Dynamical properties of the Hopfield model through bitwise arithmetic

Bastien Dumont,[1,2] David Lacoste,[3] and Michele Castellana[4]

[1] *Aix-Marseille Université, Marseille, France*

[2] *Université Paris-Saclay, Orsay, France*

[3] *Gulliver Laboratory, CNRS UMR 7083, ESPCI Paris, PSL Research University, Paris, France*

[4] *Institut Curie, PSL Research University, CNRS UMR 168, Paris, France*

(Dated: November 3, 2026)

This document contains additional information to the main paper. The content is summarized in the table of contents below.

CONTENTS

I. Method 1
A. Code structure 2
B. Naive implementation of the classic Metropolis algorithm 2

II. The Ising model 4
A. The model 4
B. Bitwise Implementation 4
C. Phase diagram 6
D. Performance test 7

III. The spin glass model 10
A. The Edwards–Anderson model 10
B. Bitwise Implementation 10
C. Performance test 11

IV. The Hopfield model 12
1. Naive implementation 12

References 14

I. METHOD

We start by presenting the broad implementation of the Metropolis algorithm, before showing how we adapt it to each specific model.

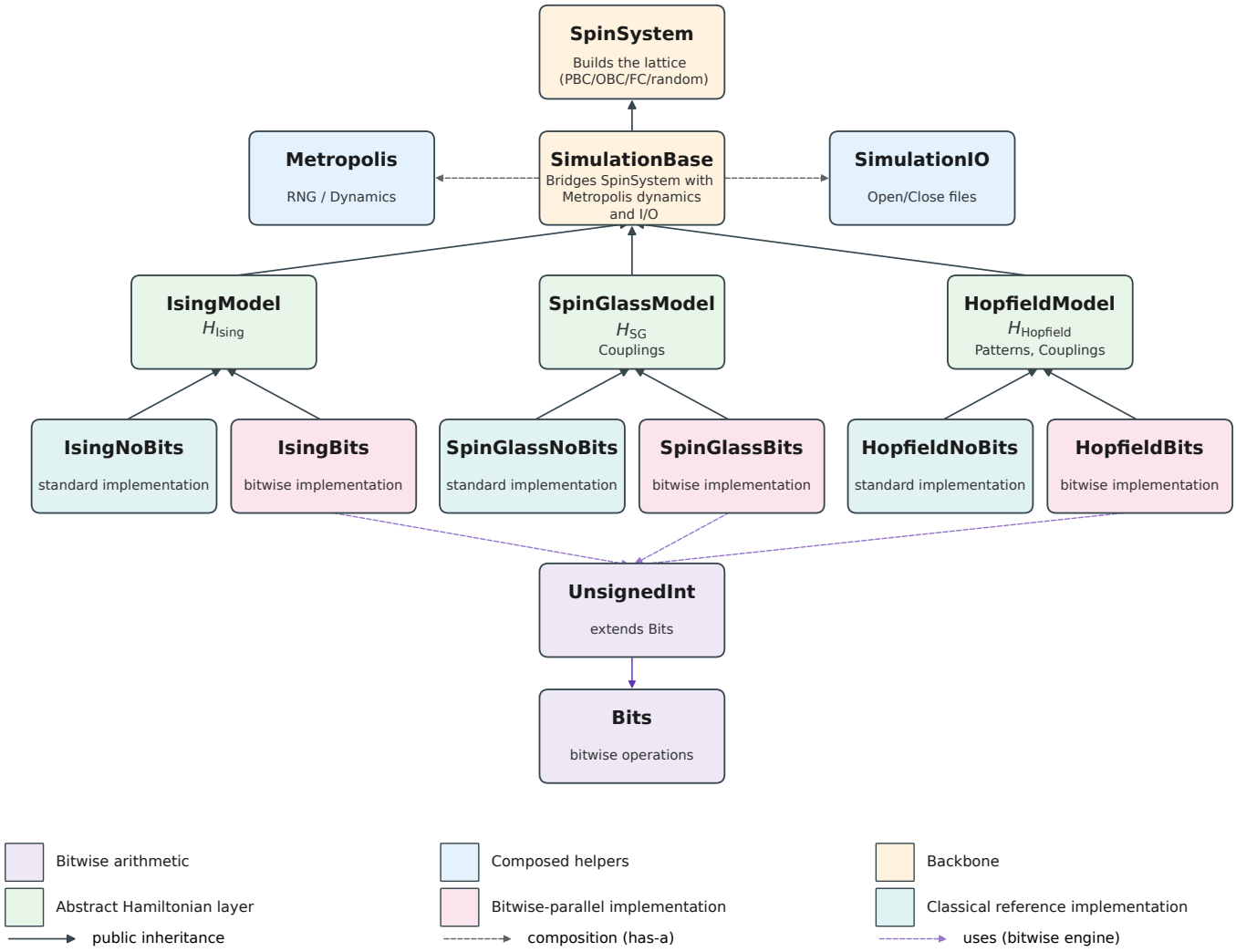


Figure 1: Code structure

A. Code structure

We here present the structure of the code. It has been entirely written in `C++`, taking advantage of the object-oriented structure to factor out the common parts for each model while enabling them to evolve depending on their specificities. Most of the time of this internship has been spent understanding the object-oriented syntax and philosophy, and implementing an easily readable and efficient code for the simulation of the Metropolis algorithm on the different models. The structure found so far is presented in Figure 1. Once the dynamics is simulated, the outputs are analyzed through python scripts. The code is built around hierarchical classes. We first implement the topology of the network, which is independent of the type of spin interaction and of the dynamical model considered. On this lattice, we set up the evolution rules (*i.e.* the Metropolis algorithm in our case), and the input/output file management. Then, the combination (topology, dynamics) is applied to one of the three models implemented (Ising, Spin glass or Hopfield) and is implemented either in its standard or bitwise version. This code can be found in [1].

B. Naive implementation of the classic Metropolis algorithm

Algorithm 1: Standard Metropolis implementation (naive)

Input: Lattice spins $\{S_i\}$, coupling J, inverse temperature β
Output: Updated lattice spins $\{S_i\}$

for sweep $= 1$ **to** N_{sweeps} **do**
  for step $= 1$ **to** N **do**
    Draw k among N
    for $r = 1$ **to** n_{bits} **do**
      Compute $\Delta_k^{(r)}$
      `// Unconditional flip: energy is lowered`
      if $\Delta_k^{(r)} \leq 0$ **then**
        $s_k^{(r)} \leftarrow -s_k^{(r)}$
      else
        `// Conditional flip: draw Poisson threshold`
        Draw q with $\theta \sim \mathcal{E}(\beta\kappa)$
        if $m \geq \Delta_k^{(r)}$ **then**
          $s_k^{(r)} \leftarrow -s_k^{(r)}$

II. THE ISING MODEL

We first implement the method on the Ising model, which is the simplest one.

A. The model

The Hamiltonian of the Ising network is given by

$$H_{\text{Ising}} = -J \sum_{\langle i,j \rangle} S_i S_j - h \sum_i S_i,$$

which, in absence of external field h becomes,

$$H_{\text{Ising},\ h=0} = -J \sum_{\langle i,j \rangle} S_i S_j. \tag{1}$$

When flipping the spin S_k ($S_k \to -S_k$), the associated energy shift is

$$\begin{aligned} \Delta E_k &= H_{\text{Ising}}\left[-S_k\right] - H_{\text{Ising}}\left[S_k\right] \\ &= 2JS_k \sum_{i \in \partial k} S_i, \end{aligned} \tag{2}$$

where $i \in \partial k$ denotes the summation over the neighbors of k. Thus, here, $\kappa = 2J$. We now need to implement the flip condition $m \geq \Delta_k$ in a bitwise formalism in order to properly parallelize the evolutions.

B. Bitwise Implementation

Remembering that S_i is actually an array containing 64 replicas of the same spin, we write $S_i = -1 + 2B_i$, with B_i a `Bits`. The sum hence becomes:

$$\begin{aligned} \sum_{i \in \partial k} S_k S_i &= \sum_{i \in \partial k} (-1 + 2B_k)(-1 + 2B_i) \\ &= \sum_{i \in \partial k} 1 - 2(B_k + B_i) + 4B_k.B_i, \end{aligned}$$

We show in Table I that the term in the sum can be re-written using the XNOR ($\odot$) logical operator.

b_k	b_i	$1 - 2(b_k + b_i) + 4b_k.b_i$	$b_k \odot b_i$	$2b_k \odot b_i - 1$
0	0	1	1	1
0	1	−1	0	−1
1	0	−1	0	−1
1	1	1	1	1

Table I: The summed term can be simplified using a XNOR operator.

Thus, the term in the sum is simply equal to $2B_k \odot B_i - 1$, where $\odot$ denotes the bitwise XNOR operation applied to the corresponding bits of each `Bits`. The sum can thus be written:

$$\sum_{i \in \partial k} S_k S_i = 2 \sum_{i \in \partial k} C_{ki} - n_k,$$

with $C_{ki} = B_k \odot B_i$ and n_k is the number of neighbor of the spin k. The bitwise flip condition is then:

$$q + n_k \geq 2 \sum_{i \in \partial k} C_{ki}. \tag{3}$$

Moreover, the integer escape flip condition (**??**) (which translates in $m \geq n_k$) constrains the range of values taken by q and $2\sum_{i\in\partial k} C_{ki}$ to n_k and $2n_k$, respectively (with $\Delta_k^{\max} = n_k$ in the case of the Ising network). This is particularly convenient, as it allows the `UnsignedInt` objects to be allocated with static sizes at initialization, removing the need for dynamic resizing during the simulation. In addition, this integer flip condition can be applied to the bitwise algorithm and avoids the bitwise flipping test calculations for each replica: when it is met, all replicas are flipped simultaneously. A simple integer comparison suffices.

The pseudocode for the bitwise implementation of the Metropolis algorithm on this Ising network is presented in the algorithm 2.

Algorithm 2: Bitwise Ising Metropolis

Input: Bit-replicated spins $\{B_i\}$, coupling J, inverse temperature β
Output: Updated lattice spins $\{S_i\}$

Variables: `Bits xnor, mask`
`UnsignedInt` T_k, `threshold`

for sweep $= 1$ **to** N_{sweep} **do**
  for step $= 1$ **to** N **do**
    Draw k among N
    Draw q with $\theta \sim \mathcal{E}(2\beta J)$
    if $m \geq n_k$ **then**
      $B_k \leftarrow \neg B_k$ `// escape condition: unconditional flip across all replicas`
    else
      $T_k \leftarrow \mathbf{0}$
      `threshold` $\leftarrow \mathbf{0}$
      for $i \in \partial k$ **do**
        `xnor` $\leftarrow B_k \odot B_i$ `// bitwise XNOR across replicas`
        T_k `+= xnor` `// increment agreement count per replica`
      $T_k \leftarrow 2T_k$
      `threshold` $\leftarrow q + n_k$
      `mask` $\leftarrow (T_k \leq$ `threshold`$)$ `// per-replica flip condition`
      $B_k \leftarrow B_k \oplus$ `mask` `// bitwise spin flip (XOR operation): only replicas`
      `// for which mask`$^{(\ell)} = 1$ `are flipped`

C. Phase diagram

We now need to make sure the bitwise implementation is consistent. This is firstly done by checking that it yields exactly the same spin configurations as the standard implementation with shared random numbers when starting from the exact same initial condition and using the same generative seed. Secondly, we can plot the phase diagram of the model and verify that it is in agreement with the known analytical results. We show in Figure 2 the obtained phase diagram for several sizes of two-dimensional square lattices with periodic boundary conditions. This phase diagram is obtained by successive cooling the system: starting from the highest temperature, we progressively decrease it, using the final state of each step as the initial configuration for the next. This reduces the computational cost compared to reinitializing the network randomly at each temperature point, since the equilibrium state at each temperature lies close to that of the previous one, requiring fewer sweeps to reach thermalization. It aslo prevents the system to get stuck into metastable states such as the strip state for which several (usually two) regions of opposite magnetizations arise when the system is initialized randomly at a low temperature.

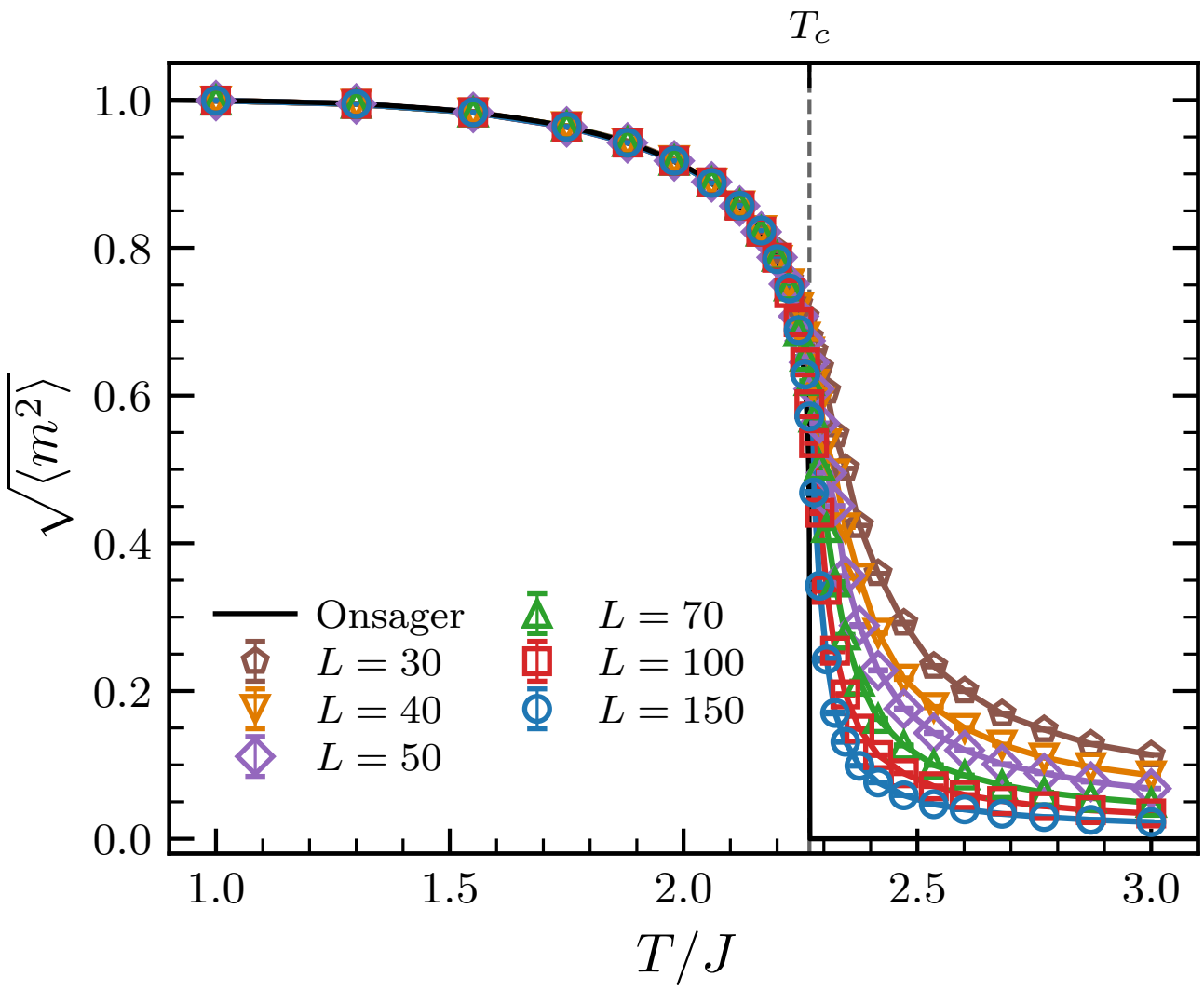


Figure 2: Phase diagram of the Ising model for several sizes of square lattices. The network is built with Periodic Boundary Conditions (PBC). The black line corresponds to the exact analytical solution found by Onsager in the case of an infinite square lattice [2]: $m(T) = 0$ for $T \geq T_c$ and $m(T)_{\pm} = \pm \left[1 - \sinh^{-4}(2K)\right]^{1/8}$ for $T \leq T_c$, with $K = J/T$ and $T_c/J = 2/\log\left(1 + \sqrt{2}\right) \approx 2.269$.

Qualitatively, it appears that the magnetization curves get closer to the exact solution when the lattice size is increased. This was to be expected as we gradually get closer to the infinite size case. We also notice an excellent agreement between the exact analytical magnetization and the obtained ones in the ferromagnetic part of the phase diagram. In this plot, each point corresponds to the mean over the $n_{\text{bits}} = 64$ independent realizations. We show in Figure 3 the trajectories over time of the magnetizations for those 64 realizations at a given temperature.

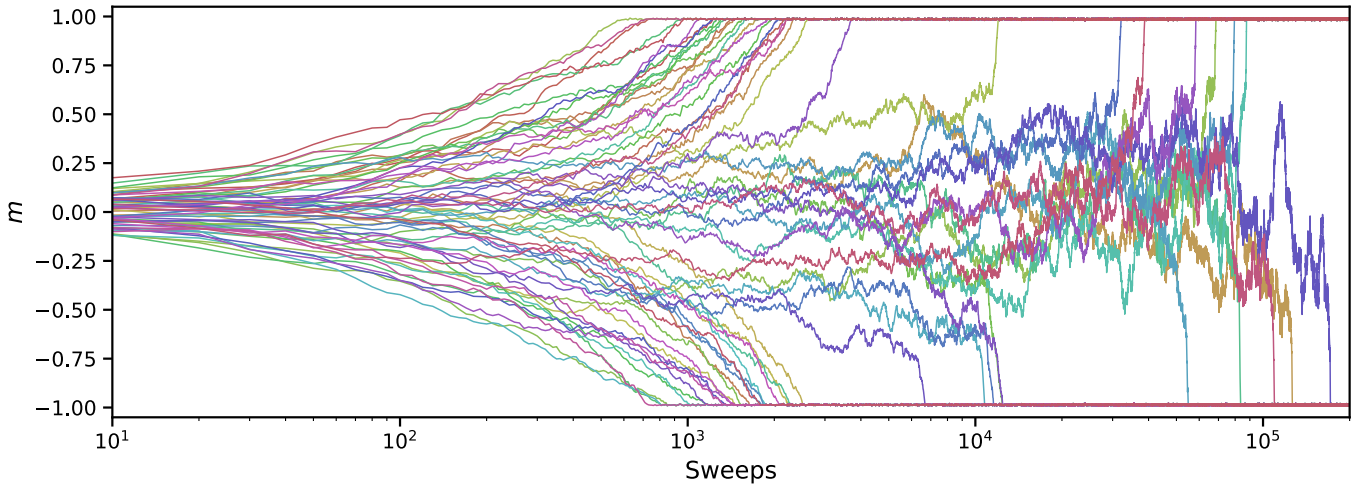


Figure 3: Trajectories of the 64 magnetizations over time for $L = 100$ and $T/J = 1.5$. All those trajectories have been obtained from the same simulation. While some realizations quickly collapse into the equilibrium attractors $m_{\pm} = \pm 0.953$, some require $\sim 10^2$ times more sweeps to reach the same equilibrium.

In order to make sure that the phase transition of the network indeed occurs at $T_c = 2.269$ for all lattice sizes, we plot the Binder cumulant U_L, defined as:

$$U_L = 1 - \frac{\langle m^4 \rangle_L}{3\langle m^2 \rangle_L^2} \tag{4}$$

Its limit values are $U_L(0) = 2/3$ and $U_L(T \to \infty) = 0$. Most importantly, it has been shown that all Binder curves for different lattice sizes L intersect at the same scale-invariant critical point (T_c, U^*) [3]. This is because at T_c, the system becomes scale invariant, making the Binder cumulant independent of the system size at this specific temperature. We show in Figure 4 the plot of the Binder cumulants for the different lattice sizes. We obtain the expected curves, with the known asymptotic values and the crossing of the curves in (T_c, U^*), which validates our implementation.

D. Performance test

Now that our algorithm has been shown to produce correct results, we wish to estimate its performance compared to the usual naive implementation. To this end, we measure the ratio of the computational time required to let the system evolve over 2^{16} sweeps using both the classical naive and the bitwise implementation, for various square lattice sizes and temperatures. The result of this performance test is shown in Figure 5.

The speedup factor lies between 20 and 45, which is a substantial performance increase. We notice that this speedup factor consistently increases with the temperature. As shown in (c), the early increase is mostly the effect of the ordering of the configurations at low temperature: before the phase transition, the state collapses into an ordered phase where all spins are aligned. This makes the unconditional flip condition $\Delta E_k \leq 0$ more frequent in the classical implementation, which is computationally cheaper, thus making it faster. This unconditional flip condition however becomes less probable when we increase the temperature, making the normalised time of the classic implementation and the speedup factor increase (especially near the critical temperature T_c) up to a certain value where the normalised time stops increasing, probably as the unconditional flip condition is never met anymore. We also notice that the unconditional flip condition in the bitwise implementation ($m \geq n_k$) becomes more and more probable with the increasing temperature, making the bitwise implementation faster. These two effects combined explain the overall shape of the speedup factor evolution, where the first increase is coming from the classical implementation and the

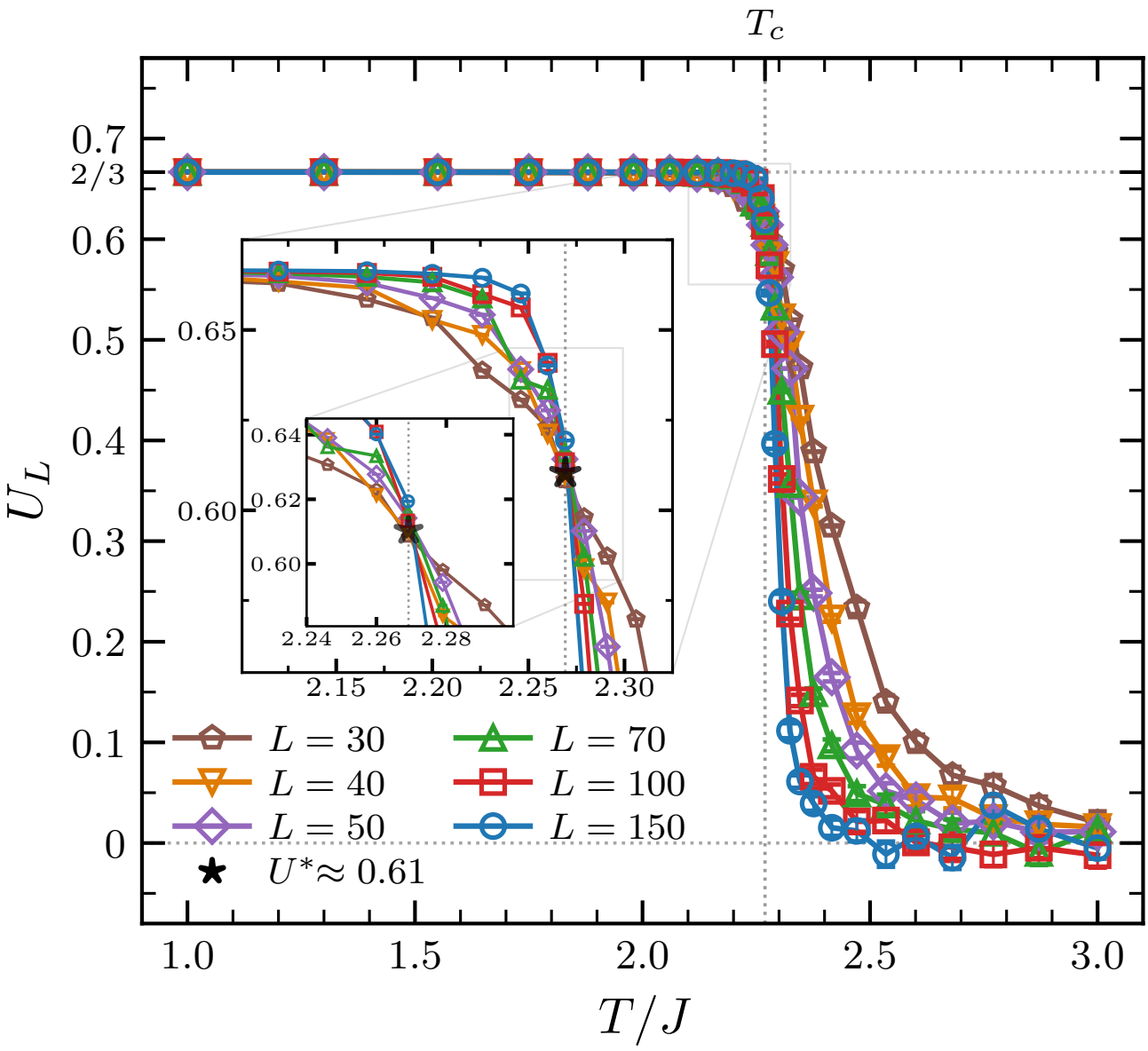
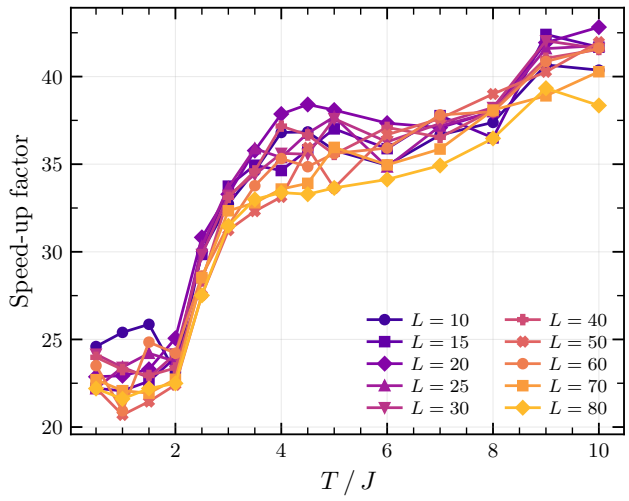
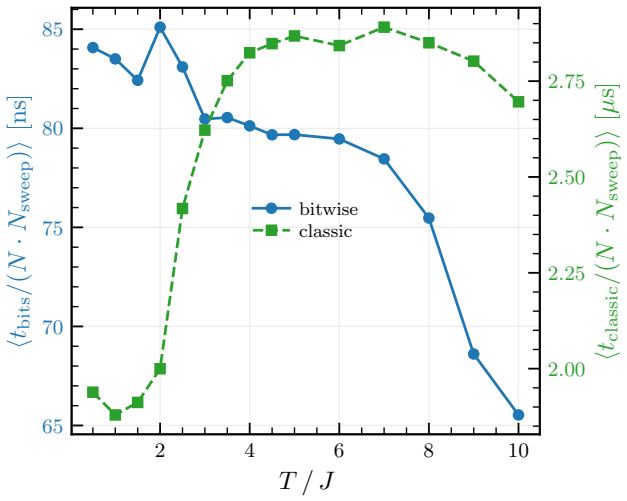


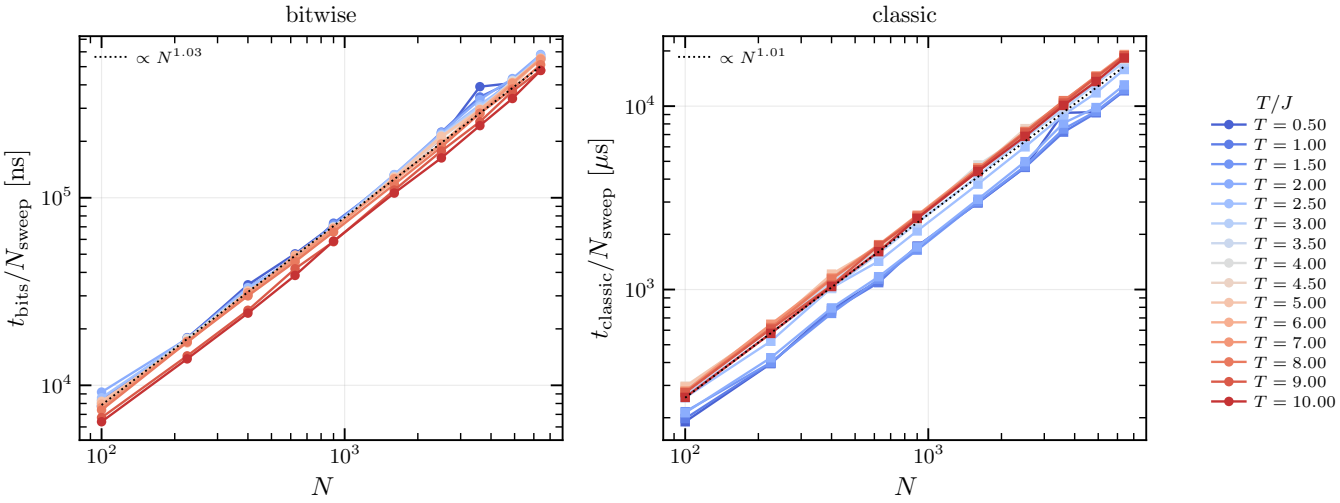
Figure 4: Binder cumulant for the different lattice sizes. The curves approach the two asymptotic limits and intersect at (T_c, U^*), as expected. Insets show the zoomed region around the critical point.



(a) Speedup factor evolution.



(b) Normalized time evolution.



(c) Sweep time depending on the lattice size for all temperatures.

Figure 5: Performance analysis of the bitwise and classical Ising Metropolis implementations. Subfigure (a) shows the evolution of the speedup factor with the temperature for several sizes of square lattices. (b) shows the evolution of the normalized spin update time with the temperature, averaged over the different lattice sizes. Finally, (c) presents the evolution of the time taken to perform one sweep depending on the lattice size, for different temperatures.

second one is the effect of the bitwise. It should be noted that this overall tendency may not be observed in the case of a comparison with the standard Metropolis algorithm, as the non-naive standard Metropolis implementation possesses the same escape condition $m \geq n_k$, making it faster and thus slowing down (or supressing) the second increase in the speedup factor.

The increase seems to be uncorrelated with the size of the system. In theory, the speedup factor can even be larger than 64, as the structural optimization alone can potentially yield a speedup of 64 by itself, as it enables us to make 64 realizations evolve in parallel. However, this structural optimization comes with the cost of creating and storing the bitwise objects used to parallelize the evolution, thus reducing the net performance improvement.

The Subfigure (c) shows that the time required grows proportionally to the system size for both implementation. The complexity of the bitwise implementation thus seems to be $\mathcal{O}\left(N_{\text{sweeps}} \cdot N \cdot \langle n_{\text{neighbor}} \rangle\right)$. It must be noted that, while we here measure an increase in the speed of the code for a fixed number of outputs, the true advantage of the bitwise implementation is to produce more independent results within the same computational time.

III. THE SPIN GLASS MODEL

We now move on a more complex model, in order to approach the structure of the Hopfield network.

A. The Edwards–Anderson model

The Edwards–Anderson (EA) model – first introduced by Edwards and Anderson in 1975 [4] – is a finite dimensional spin glass model with short range interaction. It is not a mean field spin glass model with infinite range interactions such as the Sherrington–Kirkpatrick model [5]. The model is formally described as follows:

$$H_{\mathrm{EA}} = \sum_{\langle i,j \rangle} J_{ij} S_i S_j. \tag{5}$$

Where J_{ij} is a random variable taking its values in $\{-1, 1\}$ with equal probability. Thus, it can easily be represented by a `Bits`. Under a spin flip ($S_k \to -S_k$), the energy shift is:

$$\Delta E_k = 2 S_k \sum_{i \in \partial k} J_{ki} S_i. \tag{6}$$

The structure being really similar to that of (2), we can perform the same derivation as for the Ising model and find the spin flip condition:

$$q \geq S_k \sum_{i \in \partial k} J_{ki} S_i, \tag{7}$$

with this time $\theta \sim \mathcal{E}(2\beta)$.

B. Bitwise Implementation

We now need to express the product $J_{ik} S_k S_i$ in a binary way. As shown in the previous section, the product $S_k S_i$ can be re-written $(-1 + 2B_k)(-1 + 2B_i) = -1 + 2C_{ki}$ with $C_{ki} = B_k \odot B_i$. Thus, by writing $J_{ki} = -1 + 2K_{ki}$ with K_{ki} a `Bits`, we have:

$$\begin{aligned} J_{ik} S_k S_i =& (-1 + 2K_{ki})(-1 + 2C_{ki}) \\ =& -1 + 2D_{ki}, \end{aligned}$$

with $D_{ki} = K_{ki} \odot C_{ki} = K_{ki} \odot B_k \odot B_i = K_{ki} \oplus B_k \oplus B_i$. As in the previous section, the flip condition is then:

$$q + n_k \geq 2 \sum_{i \in \partial k} D_{ki}. \tag{8}$$

Both the bitwise and the classic implementation are thus similar to this of the Ising model, and we will not present them further in this report. The bitwise algorithm for the spin glass model can be found in the appendix, see Algorithm 3. The bitwise implementation is not validated here, as the bitwise framework was already tested in the previous section, and the main interest of this model lies in bridging the complexity gap between the Ising model and the Hopfield network. We nonetheless verified that the standard and the bitwise implementations produce identical spin configurations when initialized from the same initial state and driven by the same sequence of random numbers.

Algorithm 3: Bitwise spin glass Metropolis

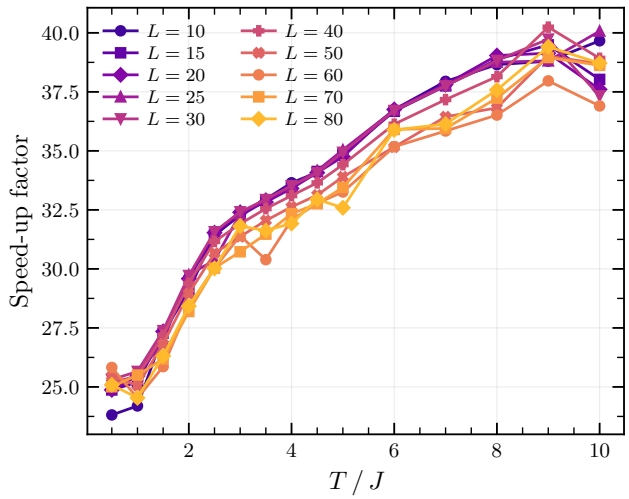
Input: Bit-replicated spins $\{B_i\}$, couplings `Couplings`, neighbor lists ∂i, inverse temperature β
Output: Updated spin configurations $\{B_i\}$

Preallocated variables: `Bits xnor, mask`
`UnsignedInt` T_i, `threshold`

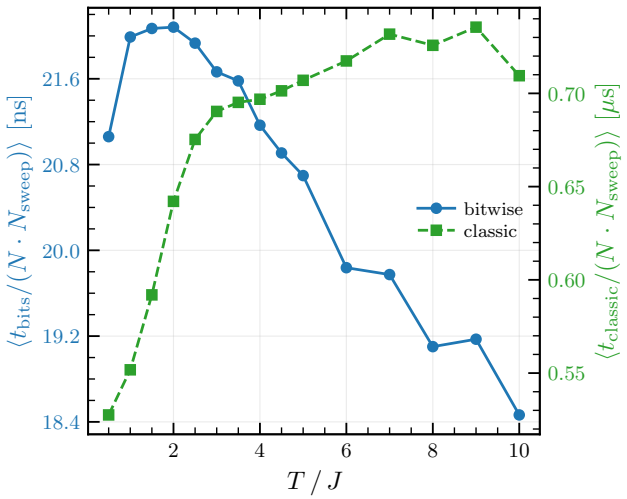
for sweep $= 1$ **to** N_{sweeps} **do**
  for step $= 1$ **to** N **do**
    Draw k among N
    Draw q with $\theta \sim \mathcal{E}(2\beta)$
    if $m \geq n_k$ **then**
      $B_k \leftarrow \neg B_k$ `// escape condition: unconditional flip across all replicas`
    else
      $T_k \leftarrow \mathbf{0}$
      `threshold` $\leftarrow \mathbf{0}$
      for $i \in \partial k$ **do**
        `xnor` $\leftarrow K_{ki} \oplus B_k \oplus B_i$ `// bitwise XNOR across replicas`
        T_k += `xnor` `// increment agreement count per replica`
      $T_k \leftarrow 2T_k$
      `threshold` $\leftarrow q + n_k$
      `mask` $\leftarrow (T_k \leq$ `threshold`$)$ `// per-replica flip condition`
      $B_k \leftarrow B_k \oplus$ `mask` `// bitwise spin flip (XOR operation): only replicas`
      `// for which mask`$^{(\ell)} = 1$ `are flipped`

C. Performance test

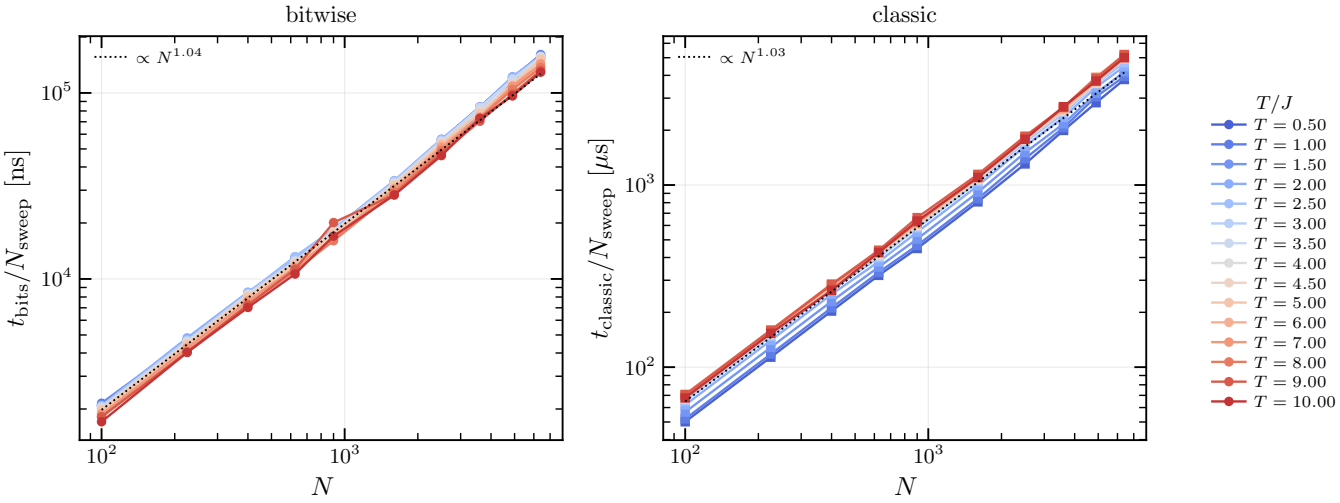
We now test the performance of the implementation, for several lattice sizes and the temperatures, this time with $N_{\text{sweeps}} = 2^{14}$. The results are shown in Figure 6. We notice the same evolution of the normalized time for both implementation as the ones observed for the Ising model. The shape of the speedup factor is similar to this of the Ising model, but its evolution is steadier, with what appears to be a saturation for the highest temperatures. The time per sweep also remains proportional to the system size, the complexities of the implementations thus should be similar to those of the Ising model.



(a) Speedup factor evolution.



(b) Normalized time evolution.



(c) Sweep time depending on the lattice size for all temperatures.

Figure 6: Performance analysis of the bitwise and classical Spin Glass Metropolis implementations.

IV. THE HOPFIELD MODEL

1. *Naive implementation*

A first naive implementation would be to unfold G_{ki} into its pattern components in order to deal with `Bits` only objects. The flip condition would then be:

$$q \geq \sum_{i \in \partial k} \sum_{\mu=1}^{p} \xi_k^\mu \xi_i^\mu S_k S_i. \tag{9}$$

As seen in the previous sections, the product $\xi_k^\mu \xi_i^\mu S_k S_i$ can simply be expressed from $X_k^\mu \odot X_i^\mu \odot B_k \odot B_i$, where X_k^μ is the boolean variable associated with ξ_k^μ. This method presents the advantage to not require too much memory allocation to store the couplings G_{ki} as they are not stored before the evolution but computed on the fly in the hot

loop. However, it requires summing over the patterns, thus introducing an additional complexity of $\mathcal{O}(p) = \mathcal{O}(\alpha N)$ per spin update, which is done $N \cdot N_{\text{sweeps}}$ times. This would result in a significant loss of performance.

Algorithm 4: Bitwise Hopfield Metropolis using shifted overlaps

Input: Bit-replicated spins $\{B_i\}$, patterns $\{X_i^\mu\}$, shifted overlaps $\{\tilde{M}^\mu\}$, shifted couplings $\{\tilde{G}_{ij}\}$, neighbor lists ∂i, number of patterns P

Output: Updated spin configurations $\{B_i\}$

Preallocated variables:
`Bits` C_{ij}, C_i^μ, `mask`
`UnsignedInt` Σ_C, Σ_{CM}, Σ_{CG}, Σ_M, LHS, RHS

for $\mu = 1$ **to** P **do**
  $\Sigma_M \leftarrow \Sigma_M + \tilde{M}^\mu$

`// Precompute shifted overlap sum`

for $i = 1$ **to** N **do**
  $\Sigma_G(i) \leftarrow 0$
  for $j \notin \partial i,\ j \neq i$ **do**
    $\Sigma_G(i) \leftarrow \Sigma_G(i) + \tilde{G}_{ij}$

`// Precompute shifted non-neighbor coupling sums`

for sweep $= 1$ **to** N_{sweeps} **do**
  for step $= 1$ **to** N **do**
    Draw random neuron k
    Draw q with $\theta \sim \mathcal{E}(2\beta/N)$

    if $m \geq pn_k$ **then**
      for $\mu = 1$ **to** P **do**
        $C_k^\mu \leftarrow B_k \odot X_k^\mu$
        $\tilde{M}^\mu \leftarrow \tilde{M}^\mu + 2 - 4C_k^\mu$
      $\Sigma_M \leftarrow \Sigma_M + 2P - 4\sum_{\mu=1}^P C_k^\mu$ `// Update shifted overlaps`
      $B_k \leftarrow \neg B_k$

    else
      $\Sigma_C \leftarrow 0$
      $\Sigma_{CG} \leftarrow 0$
      for $i \notin \partial k,\ i \neq k$ **do**
        $C_{ik} \leftarrow B_i \odot B_k$
        $\Sigma_C \leftarrow \Sigma_C + C_{ik}$
        $\Sigma_{CG} \leftarrow \Sigma_{CG} + C_{ik} \odot \tilde{G}_{ki}$
      $\Sigma_{CM} \leftarrow 0$
      for $\mu = 1$ **to** P **do**
        $C_k^\mu \leftarrow B_k \odot X_k^\mu$
        $\Sigma_{CM} \leftarrow \Sigma_{CM} + C_k^\mu \odot \tilde{M}^\mu$
      LHS $\leftarrow m + 2N\Sigma_C + \Sigma_M + 2\Sigma_{CG}$
      RHS $\leftarrow pn_k + 2\Sigma_{CM} + \Sigma_G(k) + 2p\Sigma_C$
      `mask` $\leftarrow$ (RHS $\leq$ LHS) `// Bitwise evaluation of Eq. (??)`
      for $\mu = 1$ **to** P **do**
        $C_k^\mu \leftarrow C_k^\mu \wedge$ `mask`
        $\tilde{M}^\mu \leftarrow \tilde{M}^\mu + 2$`mask` $- 4C_k^\mu$
      $\Sigma_M \leftarrow \Sigma_M + 2P$`mask` $- 4\sum_{\mu=1}^P C_k^\mu$`mask`
      $B_k \leftarrow B_k \oplus$ `mask`

[1] Bastien Dumont and Michele Castellana. Hopfield code [bastien branch]. https://github.com/Michele-s-team/hopfield/tree/Bastien.
[2] Lars Onsager. Crystal statistics. I. A two-dimensional model with an order-disorder transition. *Physical Review*, 65:117–149, 1944.
[3] K. Binder. Finite size scaling analysis of Ising model block distribution functions. *Zeitschrift für Physik B*, 43:119–140, 1981.
[4] S F Edwards and P W Anderson. Theory of spin glasses. *Journal of Physics F: Metal Physics*, 5(5):965, may 1975.
[5] David Sherrington and Scott Kirkpatrick. Solvable model of a spin-glass. *Phys. Rev. Lett.*, 35:1792–1796, Dec 1975.